敏感词过滤集成

一、系统概述

本系统基于 **DFA (确定有限状态自动机)** 算法实现敏感词过滤功能,支持从数据库动态加载敏感词,提供高效的敏感词检测、替换和标记功能。

核心特性

- 基于 DFA 算法,时间复杂度 O(n),高效处理大量文本
- 支持从数据库动态加载和刷新敏感词库
- 支持敏感词检测、替换、标记等多种处理方式
- 提供 REST API 接口,易于集成

常见敏感词过滤方案对比

方案	算法	时间复杂度	空间复杂度	适用场景	优缺点
暴力匹配	遍历 + `contains`	O(n × m × k)	O(1)	敏感词极少	❌ 性能差，效率低，无法处理大量敏感词
正则表达式	正则引擎	O(n × m)	O(m)	复杂规则匹配	⚠ 灵活但慢，适用于复杂的匹配规则
DFA 算法	确定有限状态机	O(n)	O(m × k)	大量敏感词	☑ 高效，适合处理大量敏感词
AC 自动机	Aho-Corasick	O(n + p)	O(m × k)	海量敏感词	☑ 最优方案，适用于海量敏感词匹配
布隆过滤器	Bloom Filter	O(k)	O(m)	快速判断存在	⚠ 有误判率，适用于快速判断是否包含敏感词

n: 文本的长度

m: 敏感词数量

k: 敏感词的平均长度

p: 匹配的敏感词数量

二、DFA 算法原理

DFA 算法通过构建树状结构(字典树)来匹配敏感词,核心思想:

1. **构建敏感词树**: 将所有敏感词构建成树状结构
2. **文本匹配**: 遍历文本时,沿着树进行匹配
3. **高效性**: 只需遍历一次文本,时间复杂度为 O(n)

示例:

```
敏感词: ["傻瓜", "傻逼", "白痴"]
构建的树结构:
    root
     ├─ 傻
     |    ├─ 瓜 (end)
     |    └─ 逼 (end)
     └─ 白
          └─ 痴 (end)
```

三、核心代码实现

3.1 数据库实体类

```java
@Entity
@Table(name = "sensitive_word", indexes = {
    @Index(name = "idx_category", columnList = "category"),
    @Index(name = "idx_level", columnList = "level"),
    @Index(name = "idx_enabled", columnList = "enabled")
})
@Data
public class SensitiveWord {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100, unique = true)
    private String word;

    @Column(length = 50)
    private String category; // 分类:政治、色情、暴力、辱骂等

    @Column(length = 20)
    private String level; // 敏感级别: LOW(低), MEDIUM(中), HIGH(高)

    @Column(name = "replace_char", length = 10)
    private String replaceChar; // 自定义替换字符,默认*

    @Column(columnDefinition = "TEXT")
    private String description; // 敏感词描述/备注

    @Column(name = "source", length = 50)
    private String source; // 来源:系统内置、用户举报、AI识别等

    @Column(nullable = false)
    private Boolean enabled = true; // 是否启用

    @Column(name = "match_type", length = 20)
    private String matchType; // 匹配类型: EXACT(精确), FUZZY(模糊)

    @Column(name = "create_by", length = 50)
    private String createBy; // 创建人

    @Column(name = "create_time")
    private LocalDateTime createTime;
```

```
    @Column(name = "update_by", length = 50)
    private String updateBy; // 更新人

    @Column(name = "update_time")
    private LocalDateTime updateTime;

    @Column(name = "hit_count")
    private Long hitCount = 0L; // 命中次数统计

    @Column(name = "last_hit_time")
    private LocalDateTime lastHitTime; // 最后命中时间
}
```

3.2 Repository 接口

```
@Repository
public interface SensitiveWordRepository extends JpaRepository<SensitiveWord,
Long> {

    // 查询所有启用的敏感词
    List<SensitiveWord> findByEnabledTrue();

    // 按分类查询
    List<SensitiveWord> findByCategoryAndEnabledTrue(String category);

    // 按级别查询
    List<SensitiveWord> findByLevelAndEnabledTrue(String level);

    // 按分类和级别查询
    List<SensitiveWord> findByCategoryAndLevelAndEnabledTrue(String category,
String level);

    // 检查敏感词是否存在
    boolean existsByWord(String word);

    // 查询命中次数排行
    List<SensitiveWord> findTop10ByEnabledTrueOrderByHitCountDesc();
}
```

3.3 核心过滤器类

```
@Component
public class SensitiveWordFilter {

    // DFA 算法的树结构根节点
    private Map<Character, WordNode> rootMap = new HashMap<>();

    // 结束标记
    private static final String END_FLAG = "ending";

    /**
     * 内部节点类,表示树的一个节点
     */
    private static class WordNode extends HashMap<String, Object> {
        public boolean isEnd() {
            return this.containsKey(END_FLAG);
```

```
        }

        public void setEnd(boolean isEnd) {
            if (isEnd) {
                this.put(END_FLAG, true);
            } else {
                this.remove(END_FLAG);
            }
        }
    }

    /**
     * 初始化敏感词树
     */
    public void initWordTree(List<String> sensitiveWords) {
        rootMap.clear();
        for (String word : sensitiveWords) {
            if (word == null || word.trim().isEmpty()) {
                continue;
            }
            addWord(word.trim());
        }
    }

    /**
     * 添加单个敏感词到树中
     */
    private void addWord(String word) {
        Map<String, Object> currentMap = rootMap;

        for (int i = 0; i < word.length(); i++) {
            char c = word.charAt(i);

            // 获取或创建当前字符节点
            WordNode node = (WordNode) currentMap.get(String.valueOf(c));
            if (node == null) {
                node = new WordNode();
                currentMap.put(String.valueOf(c), node);
            }

            currentMap = node;

            // 最后一个字符,标记为结束
            if (i == word.length() - 1) {
                node.setEnd(true);
            }
        }
    }

    /**
     * 检查文本是否包含敏感词
     */
    public boolean contains(String text) {
        if (text == null || text.isEmpty()) {
            return false;
        }

        for (int i = 0; i < text.length(); i++) {
```

```
            int length = checkWord(text, i);
            if (length > 0) {
                return true;
            }
        }
        return false;
    }

    /**
     * 获取文本中所有敏感词
     */
    public List<String> getSensitiveWords(String text) {
        List<String> words = new ArrayList<>();
        if (text == null || text.isEmpty()) {
            return words;
        }

        for (int i = 0; i < text.length(); i++) {
            int length = checkWord(text, i);
            if (length > 0) {
                words.add(text.substring(i, i + length));
                i += length - 1; // 跳过已匹配的部分
            }
        }
        return words;
    }

    /**
     * 替换敏感词
     */
    public String replace(String text, char replaceChar) {
        if (text == null || text.isEmpty()) {
            return text;
        }

        StringBuilder result = new StringBuilder(text);
        for (int i = 0; i < text.length(); i++) {
            int length = checkWord(text, i);
            if (length > 0) {
                for (int j = 0; j < length; j++) {
                    result.setCharAt(i + j, replaceChar);
                }
                i += length - 1;
            }
        }
        return result.toString();
    }

    /**
     * 标记敏感词(用HTML标签包裹)
     */
    public String mark(String text, String startTag, String endTag) {
        if (text == null || text.isEmpty()) {
            return text;
        }

        StringBuilder result = new StringBuilder();
        int lastIndex = 0;
```

```
        for (int i = 0; i < text.length(); i++) {
            int length = checkWord(text, i);
            if (length > 0) {
                result.append(text, lastIndex, i);
                result.append(startTag);
                result.append(text, i, i + length);
                result.append(endTag);
                lastIndex = i + length;
                i += length - 1;
            }
        }
        result.append(text.substring(lastIndex));
        return result.toString();
    }

    /**
     * 检查指定位置是否匹配敏感词
     * @return 匹配的敏感词长度,0表示不匹配
     */
    private int checkWord(String text, int startIndex) {
        Map<String, Object> currentMap = rootMap;
        int matchLength = 0;
        int tempLength = 0;

        for (int i = startIndex; i < text.length(); i++) {
            char c = text.charAt(i);
            Object node = currentMap.get(String.valueOf(c));

            if (node == null) {
                break;
            }

            tempLength++;
            if (node instanceof WordNode) {
                WordNode wordNode = (WordNode) node;
                if (wordNode.isEnd()) {
                    matchLength = tempLength;
                }
                currentMap = wordNode;
            } else {
                break;
            }
        }

        return matchLength;
    }
}
```

3.4 服务层

```
@Service
@Slf4j
public class SensitiveWordService {

    @Autowired
    private SensitiveWordRepository repository;
```

```
    @Autowired
    private SensitiveWordFilter filter;

    /**
     * 初始化敏感词库(系统启动时调用)
     */
    @PostConstruct
    public void init() {
        refreshSensitiveWords();
        log.info("敏感词库初始化完成");
    }

    /**
     * 从数据库刷新敏感词库
     */
    public void refreshSensitiveWords() {
        List<SensitiveWord> words = repository.findByEnabledTrue();
        List<String> wordList = words.stream()
            .map(SensitiveWord::getWord)
            .collect(Collectors.toList());
        filter.initWordTree(wordList);
        log.info("敏感词库已刷新,共加载 {} 个敏感词", wordList.size());
    }

    /**
     * 检查文本是否包含敏感词
     */
    public boolean checkText(String text) {
        return filter.contains(text);
    }

    /**
     * 过滤文本(替换敏感词)
     */
    public String filterText(String text) {
        return filter.replace(text, '*');
    }

    /**
     * 获取文本中的敏感词列表
     */
    public List<String> findSensitiveWords(String text) {
        return filter.getSensitiveWords(text);
    }

    /**
     * 标记敏感词
     */
    public String markSensitiveWords(String text) {
        return filter.mark(text, "<mark>", "</mark>");
    }

    /**
     * 添加敏感词
     */
    @Transactional
    public void addSensitiveWord(SensitiveWord sensitiveWord) {
```

```
        if (repository.existsByWord(sensitiveWord.getWord())) {
            throw new IllegalArgumentException("敏感词已存在");
        }
        sensitiveWord.setCreateTime(LocalDateTime.now());
        sensitiveWord.setUpdateTime(LocalDateTime.now());
        if (sensitiveWord.getLevel() == null) {
            sensitiveWord.setLevel("MEDIUM");
        }
        if (sensitiveWord.getCategory() == null) {
            sensitiveWord.setCategory("OTHER");
        }
        repository.save(sensitiveWord);
        refreshSensitiveWords();
    }

    /**
     * 批量添加敏感词
     */
    @Transactional
    public void addSensitiveWords(List<SensitiveWord> words) {
        words.forEach(word -> {
            if (word.getCreateTime() == null) {
                word.setCreateTime(LocalDateTime.now());
            }
            if (word.getUpdateTime() == null) {
                word.setUpdateTime(LocalDateTime.now());
            }
            if (word.getLevel() == null) {
                word.setLevel("MEDIUM");
            }
            if (word.getCategory() == null) {
                word.setCategory("OTHER");
            }
        });
        repository.saveAll(words);
        refreshSensitiveWords();
    }

    /**
     * 更新敏感词
     */
    @Transactional
    public void updateSensitiveWord(Long id, SensitiveWord sensitiveWord) {
        SensitiveWord existing = repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("敏感词不存在"));

        if (sensitiveWord.getWord() != null) {
            existing.setWord(sensitiveWord.getWord());
        }
        if (sensitiveWord.getCategory() != null) {
            existing.setCategory(sensitiveWord.getCategory());
        }
        if (sensitiveWord.getLevel() != null) {
            existing.setLevel(sensitiveWord.getLevel());
        }
        if (sensitiveWord.getDescription() != null) {
            existing.setDescription(sensitiveWord.getDescription());
        }
```

```
        if (sensitiveWord.getEnabled() != null) {
            existing.setEnabled(sensitiveWord.getEnabled());
        }
        existing.setUpdateBy(sensitiveWord.getUpdateBy());
        existing.setUpdateTime(LocalDateTime.now());

        repository.save(existing);
        refreshSensitiveWords();
    }

    /**
     * 记录敏感词命中
     */
    @Async
    public void recordHit(String word) {
        repository.findByEnabledTrue().stream()
            .filter(sw -> sw.getWord().equals(word))
            .findFirst()
            .ifPresent(sw -> {
                sw.setHitCount(sw.getHitCount() + 1);
                sw.setLastHitTime(LocalDateTime.now());
                repository.save(sw);
            });
    }

    /**
     * 按分类获取敏感词
     */
    public List<SensitiveWord> getByCategory(String category) {
        return repository.findByCategoryAndEnabledTrue(category);
    }

    /**
     * 按级别获取敏感词
     */
    public List<SensitiveWord> getByLevel(String level) {
        return repository.findByLevelAndEnabledTrue(level);
    }

    /**
     * 获取热门敏感词(命中次数Top10)
     */
    public List<SensitiveWord> getTopHitWords() {
        return repository.findTop10ByEnabledTrueOrderByHitCountDesc();
    }

    /**
     * 删除敏感词
     */
    @Transactional
    public void deleteSensitiveWord(Long id) {
        repository.deleteById(id);
        refreshSensitiveWords();
    }
}
```

3.5 控制器

```
@RestController
@RequestMapping("/api/sensitive")
public class SensitiveWordController {

    @Autowired
    private SensitiveWordService service;

    /**
     * 检查文本
     */
    @PostMapping("/check")
    public ResponseEntity<?> checkText(@RequestBody Map<String, String> request)
{
        String text = request.get("text");
        boolean hasSensitive = service.checkText(text);
        List<String> words = service.findSensitiveWords(text);

        Map<String, Object> result = new HashMap<>();
        result.put("hasSensitive", hasSensitive);
        result.put("sensitiveWords", words);
        return ResponseEntity.ok(result);
    }

    /**
     * 过滤文本
     */
    @PostMapping("/filter")
    public ResponseEntity<?> filterText(@RequestBody Map<String, String>
request) {
        String text = request.get("text");
        String filtered = service.filterText(text);
        return ResponseEntity.ok(Map.of("filteredText", filtered));
    }

    /**
     * 标记敏感词
     */
    @PostMapping("/mark")
    public ResponseEntity<?> markText(@RequestBody Map<String, String> request)
{
        String text = request.get("text");
        String marked = service.markSensitiveWords(text);
        return ResponseEntity.ok(Map.of("markedText", marked));
    }

    /**
     * 添加敏感词
     */
    @PostMapping("/word")
    public ResponseEntity<?> addWord(@RequestBody SensitiveWord sensitiveWord) {
        try {
            service.addSensitiveWord(sensitiveWord);
            return ResponseEntity.ok(Map.of("success", true, "message", "添加成
功"));
        } catch (IllegalArgumentException e) {
```

```
            return ResponseEntity.badRequest().body(Map.of("success", false,
"message", e.getMessage()));
        }
    }

    /**
     * 更新敏感词
     */
    @PutMapping("/word/{id}")
    public ResponseEntity<?> updateWord(@PathVariable Long id, @RequestBody
SensitiveWord sensitiveWord) {
        try {
            service.updateSensitiveWord(id, sensitiveWord);
            return ResponseEntity.ok(Map.of("success", true, "message", "更新成
功"));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(Map.of("success", false,
"message", e.getMessage()));
        }
    }

    /**
     * 删除敏感词
     */
    @DeleteMapping("/word/{id}")
    public ResponseEntity<?> deleteWord(@PathVariable Long id) {
        service.deleteSensitiveWord(id);
        return ResponseEntity.ok(Map.of("success", true, "message", "删除成功"));
    }

    /**
     * 按分类查询敏感词
     */
    @GetMapping("/words/category/{category}")
    public ResponseEntity<?> getByCategory(@PathVariable String category) {
        List<SensitiveWord> words = service.getByCategory(category);
        return ResponseEntity.ok(words);
    }

    /**
     * 按级别查询敏感词
     */
    @GetMapping("/words/level/{level}")
    public ResponseEntity<?> getByLevel(@PathVariable String level) {
        List<SensitiveWord> words = service.getByLevel(level);
        return ResponseEntity.ok(words);
    }

    /**
     * 获取热门敏感词
     */
    @GetMapping("/words/top")
    public ResponseEntity<?> getTopWords() {
        List<SensitiveWord> words = service.getTopHitWords();
        return ResponseEntity.ok(words);
    }

    /**
```

```
     * 刷新敏感词库
     */
    @PostMapping("/refresh")
    public ResponseEntity<?> refresh() {
        service.refreshSensitiveWords();
        return ResponseEntity.ok("刷新成功");
    }
}
```

四、配置文件

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/your_database?
useUnicode=true&characterEncoding=utf8
    username: root
    password: your_password
    driver-class-name: com.mysql.cj.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

五、数据库表结构

```
CREATE TABLE sensitive_word (
    id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键ID',
    word VARCHAR(100) NOT NULL UNIQUE COMMENT '敏感词',
    category VARCHAR(50) DEFAULT 'OTHER' COMMENT '分类:POLITICS(政治), PORN(色情),
VIOLENCE(暴力), ABUSE(辱骂), ADVERTISING(广告), OTHER(其他)',
    level VARCHAR(20) DEFAULT 'MEDIUM' COMMENT '敏感级别: LOW(低), MEDIUM(中),
HIGH(高)',
    replace_char VARCHAR(10) DEFAULT '*' COMMENT '自定义替换字符',
    description TEXT COMMENT '敏感词描述/备注',
    source VARCHAR(50) DEFAULT 'SYSTEM' COMMENT '来源: SYSTEM(系统内置),
USER_REPORT(用户举报), AI_DETECT(AI识别), MANUAL(手动添加)',
    enabled TINYINT(1) DEFAULT 1 COMMENT '是否启用: 0-禁用, 1-启用',
    match_type VARCHAR(20) DEFAULT 'EXACT' COMMENT '匹配类型: EXACT(精确匹配),
FUZZY(模糊匹配)',
    create_by VARCHAR(50) COMMENT '创建人',
    create_time DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    update_by VARCHAR(50) COMMENT '更新人',
    update_time DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间',
    hit_count BIGINT DEFAULT 0 COMMENT '命中次数统计',
    last_hit_time DATETIME COMMENT '最后命中时间',
    INDEX idx_category (category),
    INDEX idx_level (level),
    INDEX idx_enabled (enabled),
    INDEX idx_hit_count (hit_count),
    INDEX idx_create_time (create_time)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='敏感词表';

-- 插入测试数据
INSERT INTO sensitive_word (word, category, level, description, source, create_by) VALUES
('傻瓜', 'ABUSE', 'LOW', '轻度辱骂词汇', 'SYSTEM', 'admin'),
('白痴', 'ABUSE', 'MEDIUM', '中度辱骂词汇', 'SYSTEM', 'admin'),
('垃圾', 'ABUSE', 'LOW', '轻度贬义词', 'SYSTEM', 'admin'),
('政治敏感词', 'POLITICS', 'HIGH', '政治相关敏感内容', 'SYSTEM', 'admin'),
('色情内容', 'PORN', 'HIGH', '色情相关敏感内容', 'SYSTEM', 'admin');

-- 创建敏感词命中日志表(可选,用于审计)
CREATE TABLE sensitive_word_log (
    id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键ID',
    word_id BIGINT COMMENT '敏感词ID',
    word VARCHAR(100) COMMENT '命中的敏感词',
    original_text TEXT COMMENT '原始文本(可脱敏存储)',
    user_id VARCHAR(50) COMMENT '触发用户ID',
    ip_address VARCHAR(50) COMMENT 'IP地址',
    scene VARCHAR(50) COMMENT '触发场景:评论、帖子、聊天等',
    action VARCHAR(20) COMMENT '处理动作: FILTER(过滤), BLOCK(拦截), WARN(警告)',
    create_time DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    INDEX idx_word_id (word_id),
    INDEX idx_user_id (user_id),
    INDEX idx_create_time (create_time),
    FOREIGN KEY (word_id) REFERENCES sensitive_word(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='敏感词命中日志表';
```

六、使用示例

6.1 API 调用示例

```
# 1. 检查文本
curl -X POST http://localhost:8080/api/sensitive/check \
  -H "Content-Type: application/json" \
  -d '{"text":"你这个傻瓜,真是个白痴"}'

# 2. 过滤文本
curl -X POST http://localhost:8080/api/sensitive/filter \
  -H "Content-Type: application/json" \
  -d '{"text":"你这个傻瓜,真是个白痴"}'

# 3. 添加敏感词(完整信息)
curl -X POST http://localhost:8080/api/sensitive/word \
  -H "Content-Type: application/json" \
  -d '{
    "word": "新敏感词",
    "category": "ABUSE",
    "level": "MEDIUM",
    "description": "测试敏感词",
    "source": "MANUAL",
    "createBy": "admin"
  }'
```

```
# 4. 更新敏感词
curl -X PUT http://localhost:8080/api/sensitive/word/1 \
  -H "Content-Type: application/json" \
  -d '{
    "level": "HIGH",
    "description": "更新描述",
    "updateBy": "admin"
  }'

# 5. 删除敏感词
curl -X DELETE http://localhost:8080/api/sensitive/word/1

# 6. 按分类查询敏感词
curl -X GET http://localhost:8080/api/sensitive/words/category/ABUSE

# 7. 按级别查询敏感词
curl -X GET http://localhost:8080/api/sensitive/words/level/HIGH

# 8. 获取热门敏感词(命中次数Top10)
curl -X GET http://localhost:8080/api/sensitive/words/top

# 9. 刷新敏感词库
curl -X POST http://localhost:8080/api/sensitive/refresh
```

6.2 Java 代码调用

```
@Autowired
private SensitiveWordService sensitiveWordService;

public void example() {
    String text = "你这个傻瓜,真是个白痴";

    // 检查是否包含敏感词
    boolean hasSensitive = sensitiveWordService.checkText(text);

    // 获取敏感词列表
    List<String> words = sensitiveWordService.findSensitiveWords(text);

    // 过滤敏感词
    String filtered = sensitiveWordService.filterText(text);
    // 结果: "你这个**,真是个**"

    // 标记敏感词
    String marked = sensitiveWordService.markSensitiveWords(text);
    // 结果: "你这个<mark>傻瓜</mark>,真是个<mark>白痴</mark>"
}
```

七、性能待优化项

1. **使用缓存**: 对于高频访问场景,可以使用 Redis 缓存敏感词树
2. **定时刷新**: 设置定时任务定期从数据库刷新敏感词
3. **异步处理**: 对于大批量文本处理,使用异步方式
4. **分词优化**: 结合中文分词工具提高匹配准确度

八、注意事项

1. 敏感词更新后需要调用刷新接口重新加载

九、扩展功能

9.1 敏感词配置枚举类

```
public class SensitiveWordConstants {

    // 分类枚举
    public enum Category {
        POLITICS("POLITICS", "政治"),
        PORN("PORN", "色情"),
        VIOLENCE("VIOLENCE", "暴力"),
        ABUSE("ABUSE", "辱骂"),
        ADVERTISING("ADVERTISING", "广告"),
        OTHER("OTHER", "其他");

        private String code;
        private String name;

        Category(String code, String name) {
            this.code = code;
            this.name = name;
        }
    }

    // 级别枚举
    public enum Level {
        LOW("LOW", "低", 1),
        MEDIUM("MEDIUM", "中", 2),
        HIGH("HIGH", "高", 3);

        private String code;
        private String name;
        private Integer priority;

        Level(String code, String name, Integer priority) {
            this.code = code;
            this.name = name;
            this.priority = priority;
        }
    }

    // 来源枚举
    public enum Source {
        SYSTEM("SYSTEM", "系统内置"),
        USER_REPORT("USER_REPORT", "用户举报"),
        AI_DETECT("AI_DETECT", "AI识别"),
        MANUAL("MANUAL", "手动添加");

        private String code;
        private String name;
```

```
        Source(String code, String name) {
            this.code = code;
            this.name = name;
        }
    }
}
```

9.2 敏感词导入导出功能

```
@Service
public class SensitiveWordImportExportService {

    @Autowired
    private SensitiveWordRepository repository;

    @Autowired
    private SensitiveWordService sensitiveWordService;

    /**
     * 从Excel导入敏感词
     */
    public void importFromExcel(MultipartFile file) throws IOException {
        // 使用 Apache POI 或 EasyExcel 读取Excel
        List<SensitiveWord> words = new ArrayList<>();
        // ... Excel解析逻辑
        sensitiveWordService.addSensitiveWords(words);
    }

    /**
     * 导出敏感词到Excel
     */
    public void exportToExcel(HttpServletResponse response) throws IOException {
        List<SensitiveWord> words = repository.findAll();
        // ... Excel生成逻辑
    }

    /**
     * 从文本文件批量导入(每行一个敏感词)
     */
    public void importFromText(MultipartFile file, String category, String
level) throws IOException {
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(file.getInputStream(), StandardCharsets.UTF_8)
        );

        List<SensitiveWord> words = new ArrayList<>();
        String line;
        while ((line = reader.readLine()) != null) {
            if (line.trim().isEmpty()) continue;

            SensitiveWord word = new SensitiveWord();
            word.setWord(line.trim());
            word.setCategory(category);
            word.setLevel(level);
            word.setSource("MANUAL");
            words.add(word);
        }
```

```
        sensitiveWordService.addSensitiveWords(words);
    }
}
```

9.3 定时任务配置

```
@Component
@EnableScheduling
public class SensitiveWordScheduledTask {

    @Autowired
    private SensitiveWordService sensitiveWordService;

    /**
     * 每天凌晨2点自动刷新敏感词库
     */
    @Scheduled(cron = "0 0 2 * * ?")
    public void autoRefresh() {
        sensitiveWordService.refreshSensitiveWords();
        log.info("定时任务:敏感词库已自动刷新");
    }

    /**
     * 每周统计敏感词命中情况
     */
    @Scheduled(cron = "0 0 3 ? * MON")
    public void weeklyStatistics() {
        List<SensitiveWord> topWords = sensitiveWordService.getTopHitWords();
        // 生成统计报告,发送邮件等
        log.info("每周敏感词统计完成,Top词汇:{}", topWords);
    }
}
```

9.4 待扩展功能

- **支持敏感词分级处理**: 根据不同级别采取不同处理策略
- **支持正则表达式匹配**: 对于变体词汇的检测
- **支持拼音、繁简体转换**: 防止用户通过变形规避检测
- **添加白名单机制**: 某些上下文中允许特定词汇
- **支持敏感词变体检测**: 如"傻@瓜"、"傻·瓜"等
- **集成日志记录**: 详细记录敏感词触发情况,用于审计和分析
- **支持多语言**: 不同语言的敏感词库
- **AI智能学习**: 根据用户举报自动学习新的敏感词